

Rapport de projet

Matière : Java Avancé

Rendu : 24 mai 2026

Equipe :

- HABONIMANA Patrick Pierre
- LOS Mélissandre
- MANGIN Raphaël
- IRAZIGAMA Victor-Nervy

1. Introduction.....	4
A. Contexte.....	4
B. Objectifs.....	4
C. Contraintes.....	4
2. Architecture du projet.....	5
A. Architecture du système distribué.....	5
B. Découpage en composants.....	5
C. Choix techniques.....	6
D. Flux d'une commande.....	6
3. Organisation du code source.....	8
A. Arborescence du backend.....	8
B. Arborescence du frontend.....	10
C. Diagramme de classes du projet.....	11
1. Diagramme du frontend.....	11
2. Diagramme du serveur.....	12
3. Diagramme de l'usine.....	13
4. Conception du backend.....	13
A. Explications générales.....	13
B. L'usine.....	13
1. Interface Usine.....	14
2. Classe UsineImpl.....	14
3. Classe Dispatcher.....	14
C. Serveur.....	15
1. Classe Main.....	15
2. Classe MqttGateway.....	15
3. Classe OrderHandler.....	15
4. Classe SerialChecker.....	16
5. Classe Config.....	16
5. Conception du frontend.....	16
A. Explications générales.....	16
B. Navigation entre les écrans.....	19
C. MQTT côté frontend.....	20
D. Mise à jour de l'interface depuis le thread MQTT.....	20
E. Gestion du panier.....	20
F. Gestion du timeout sur l'écran d'attente.....	20
G. Catalogue dynamique.....	21
6. Protocole MQTT.....	21
A. Conventions de nommage des topics.....	21
B. Identifiants uniques.....	21
C. Tableau récapitulatif des topics.....	22
D. QoS et rétention.....	22
E. Cas d'erreur et résilience.....	23
1. Perte de connexion broker.....	23

2. Backend indisponible.....	23
3. Payload malformé.....	23
F. Diagrammes de séquences.....	24
7. Format de Sérialisation.....	26
A. Grammaire.....	26
B. Types de messages.....	26
C. Exemples annotés.....	27
1. Passage de commande.....	27
2. Livraison.....	27
3. Erreur avec caractère réservé dans le message.....	27
D. Implémentation.....	27
8. Tests.....	28
A. Les tests réalisés.....	28
B. Test non implémentés.....	28
9. CI/CD et publication.....	29
A. Workflows.....	29
B. Secrets.....	30
10. Questions du sujet.....	30
A. Comment gérer l'absence d'usine connectée au bus pour produire les lunettes ?.....	30
B. Comment gérer plusieurs commandes en parallèle pour un même client?.....	31
C. Que faut-il modifier pour pouvoir gérer plusieurs usines ?.....	31
11. Répartition du travail.....	32
12. Conclusion.....	32

1. Introduction

A. Contexte

Le projet consiste à développer un système distribué permettant de commander, fabriquer et réceptionner des lunettes connectées. Il y a 4 modèles de lunettes dans le catalogue. La communication entre l'application clientes et le backend se fait via le protocole MQTT (Message Queuing Telemetry Transport).

B. Objectifs

Ce projet a de nombreux objectifs, autant techniques que fonctionnels:

- L'application doit permettre à plusieurs clients de passer des commandes de manières simultanés.
- On doit pouvoir passer des commandes selon certaines règles comme choisir un ou plusieurs types de lunettes et une quantité pour chaque type.
- La production doit être optimisée selon la capacité de la machine (défini dans le fichier config.properties).
- Le client doit pouvoir recevoir les numéros de série de ses lunettes commandées (les numéros de série sont unique).
- Sur l'application, on doit pouvoir vérifier la validité d'un numéro de série.
- La communication entre le client et le serveur doit se faire via le protocole MQTT.
- Il doit y avoir une gestion robuste des erreurs.
- Le projet doit comprendre de la documentation.

Le projet doit également être "production-ready" c'est-à-dire qu'il doit pouvoir être lancé directement grâce à deux .jar fournis.

C. Contraintes

Le projet comporte plusieurs contraintes de structuration :

- Architecture événementielle grâce à une communication exclusive via MQTT.
- Découpage du backend en plusieurs éléments distincts : le serveur et l'usine.
- Interdiction d'utiliser des frameworks.

- Le broker Mosquitto est imposé sur le port par défaut sans authentification.
- Des GitHub action doivent déclencher la publication des .jar sur GitHub Packages.
- Utilisation obligatoire de JavaFx pour la partie visuelle.
- Utilisation de Maven.

2. Architecture du projet

A. Architecture du système distribué

Notre système est constitué de trois éléments : le frontend, le backend et le broker MQTT (Mosquitto). Ces éléments communiquent exclusivement via MQTT. Le schéma suivant montre l'architecture et la communication entre les éléments du projet.

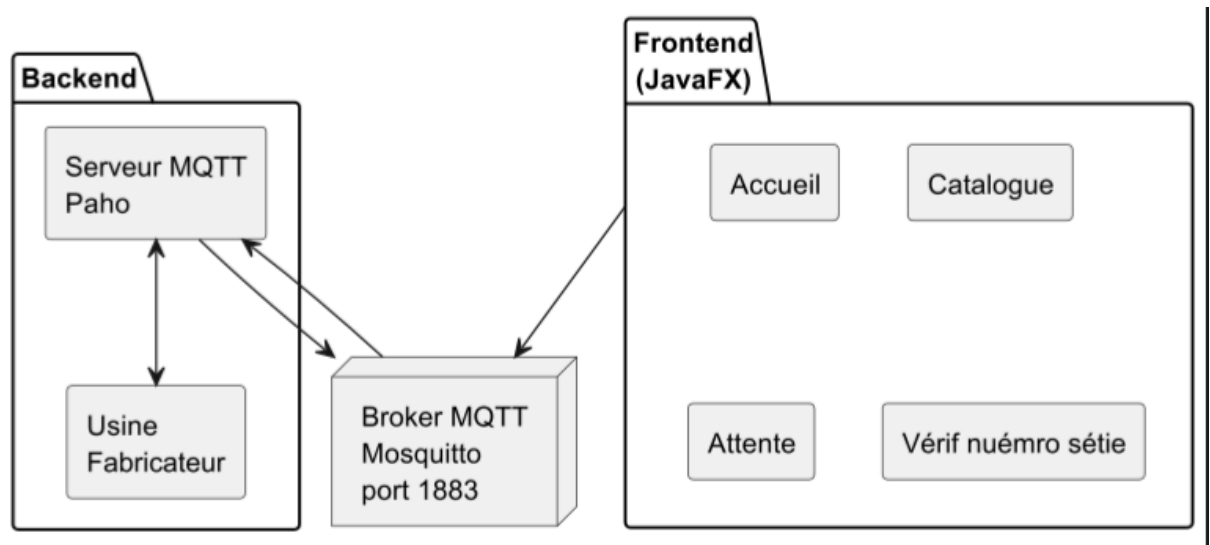


Figure 1 : Architecture du système distribué

B. Découpage en composants

Il y a 3 composants majeurs dans le projet :

- Le frontend en JavaFx : C'est la partie visuelle de l'application que le client utilise. Dedans on retrouve les différentes pages de l'application, la page d'accueil, la page de vérification des numéros de série, la page du catalogue ou encore la page de chargement. Cette partie gère tout

ce qui concerne la navigation entre ces pages, la composition du panier ou encore la publication des commandes sur MQTT. Elle gère également la réception asynchrone des réponses. L'application peut être lancée par plusieurs clients en parallèle.

- Le serveur MQTT : c'est un composant du backend. Il écoute les commandes et publie les réponses. Il s'appuie sur la bibliothèque Eclipse Paho.
- L'usine : c'est un composant du backend qui utilise le fabricant fourni. Elle sera accessible par le client sous la forme d'une API thread-safe produire(). C'est elle qui se charge du découpage des commandes en lots et de la production des lunettes.

Le serveur et l'usine sont regroupés dans le même .jar mais sont dans deux modules Maven différents. Cela peut permettre à l'usine d'être utilisée par d'autres applications.

C. Choix techniques

Elément	Version	Justification
Java	21	Fabricateur en Java 17+ et LTS disponible
JavaFx	21.0.4	LTS
MQTT client	Eclipse Paho 1.2.5	Référence Java pour MQTT version 3
Broker MQTT	Mosquitto	Imposé
Tests	JUnit 5.1.0	Librairie standard de tests unitaires.
Build	Maven 3.9	Imposé

D. Flux d'une commande

Le diagramme suivant représente le cycle de vie d'une commande, ici le broker MQTT fait la passerelle entre le frontend et le backend. Le cycle de vie d'une commande s'articule selon les étapes suivantes:

- Le frontend publie la commande sur le topic orders/id_commande
- Le backend la valide et publie sur orders/id_commande/validated

- Le backend informe le client que la commande est en cours de traitement en publiant sur `orders/id_commande/processing`.
- La production est lancée
- Une fois la production terminée, le backend publie le statut complété sur `order/id_commande/processed`
- En enfin pour la livraison, le backend publie sur `orders/id_commande/delivery`

Le client doit obligatoirement être abonné aux topics sur lesquels le backend publie et inversement.

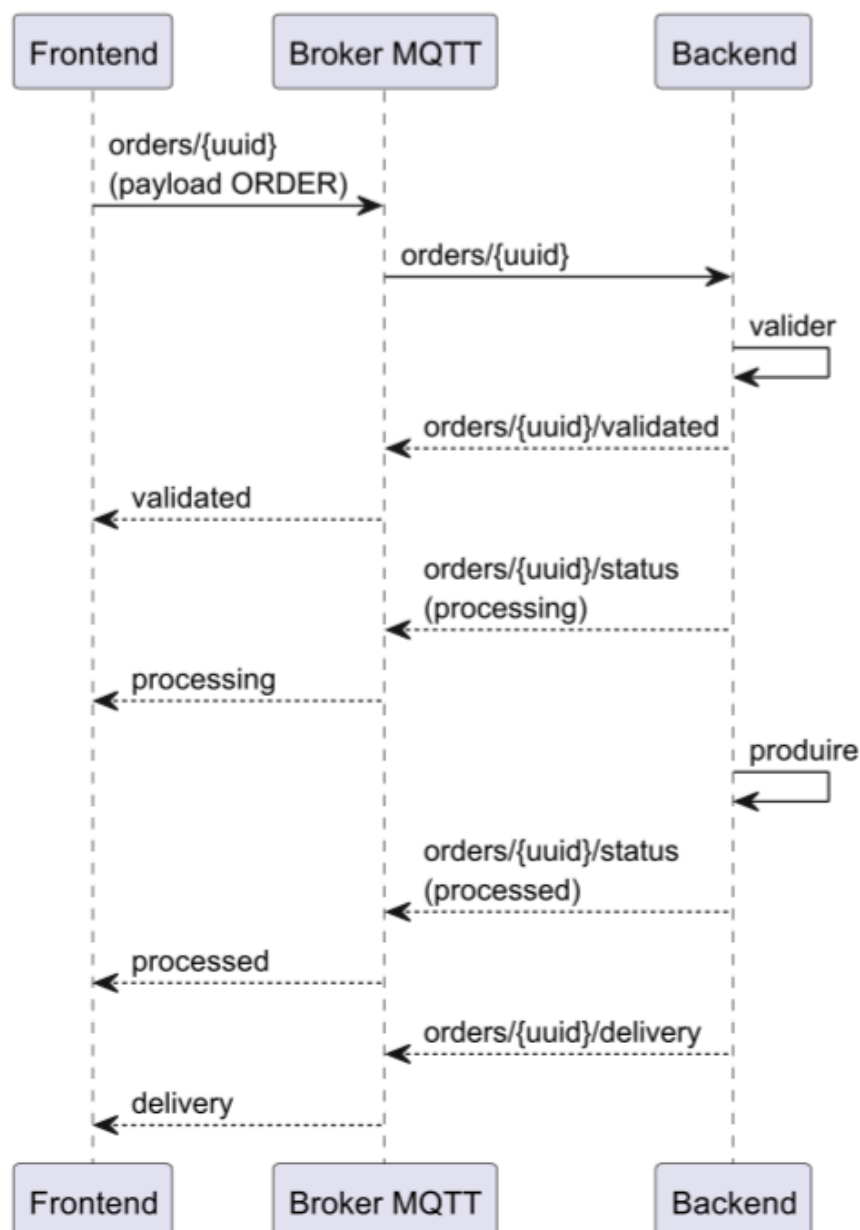
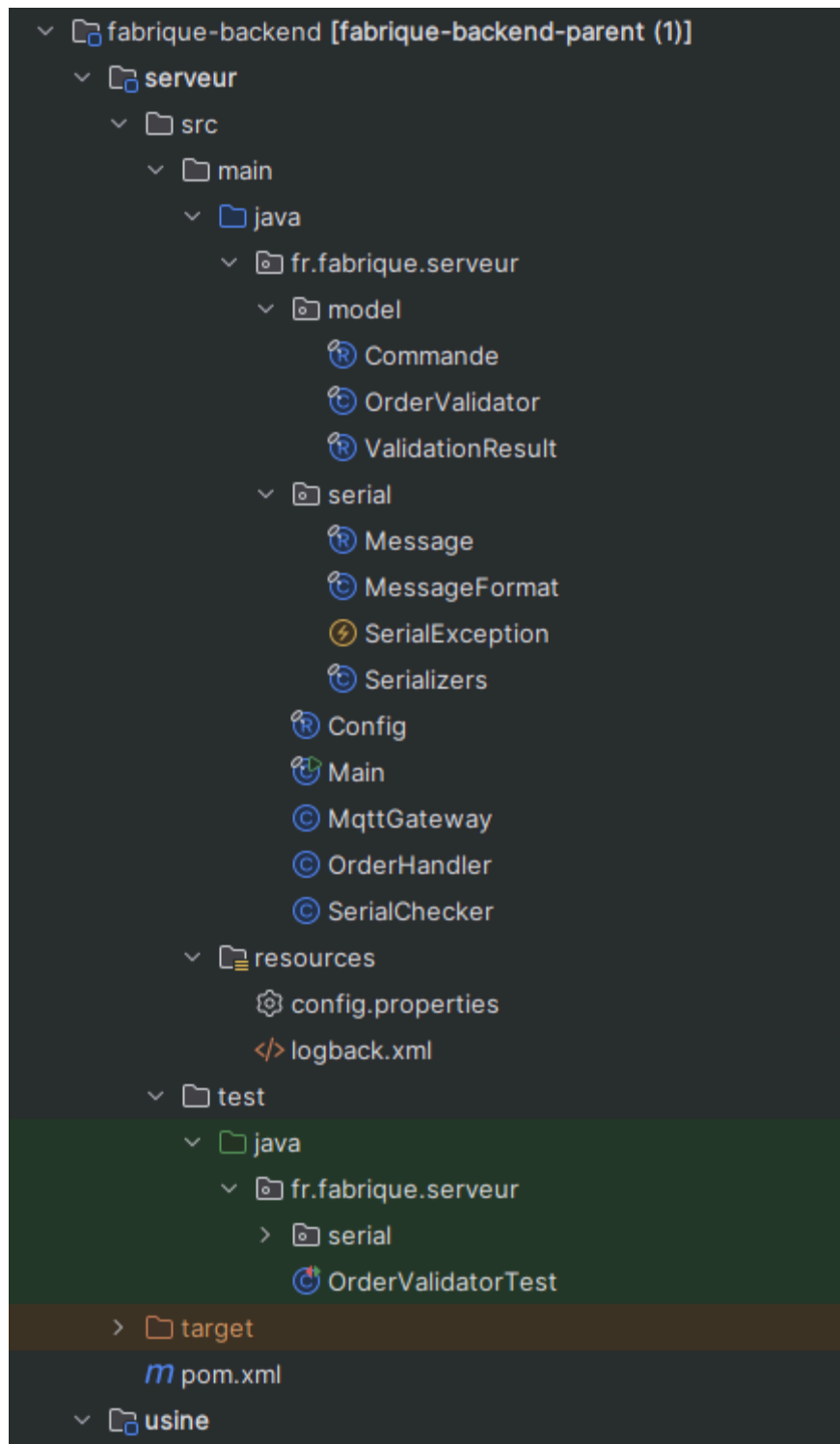


Figure 2 : Cycle de vie d'une commande

3. Organisation du code source

A. Arborescence du backend



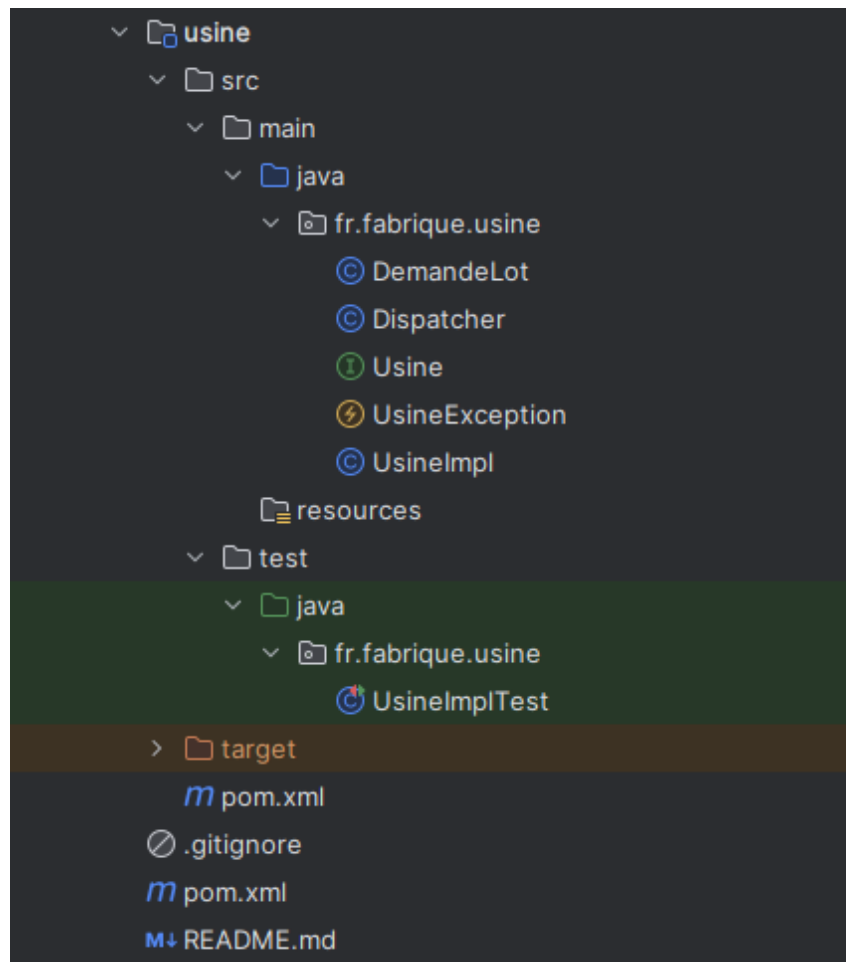


Figure 3 : Arborescence de la partie backend

B. Arborescence du frontend

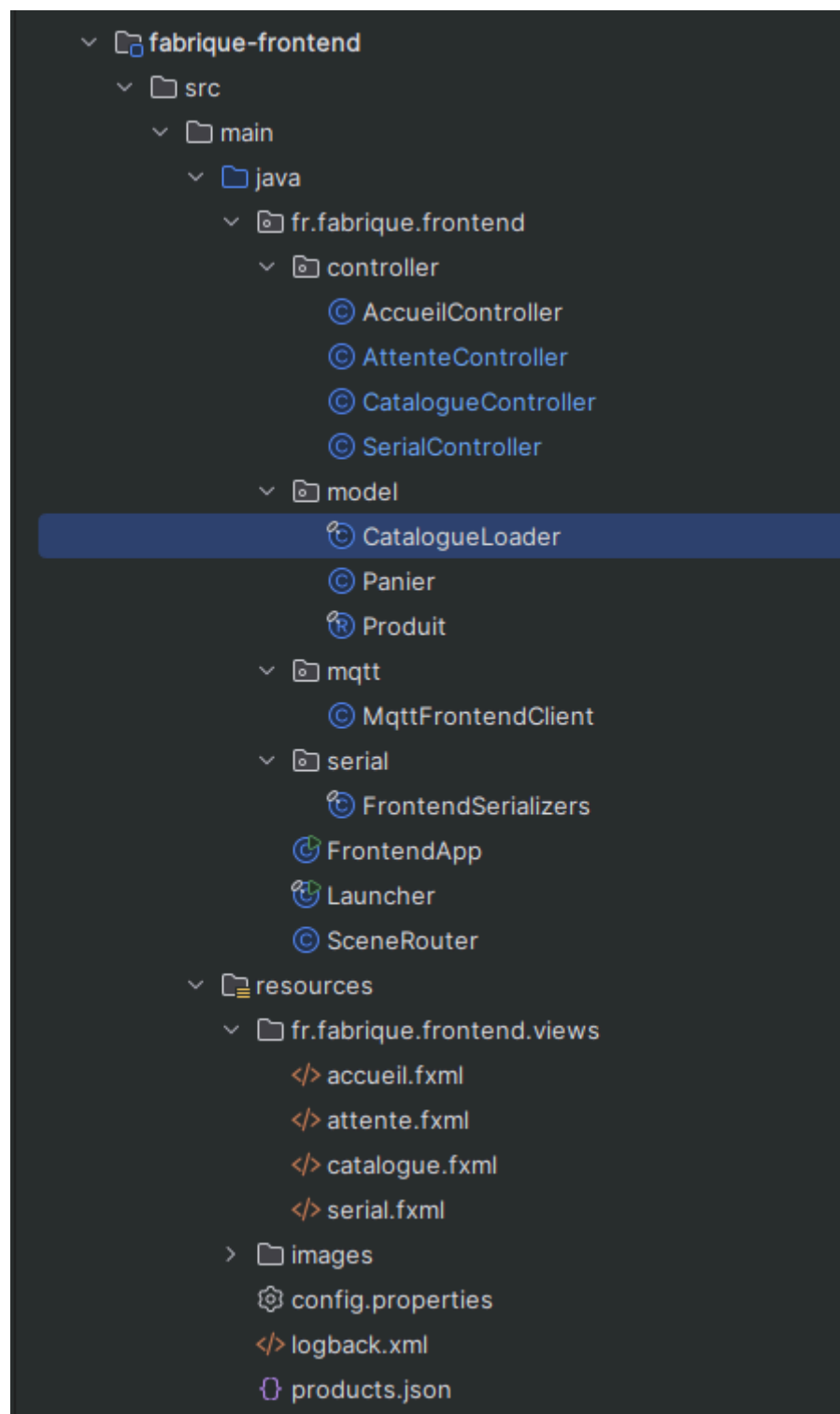


Figure 4 : Arborescence de la partie frontend

C. Diagramme de classes du projet

1. Diagramme du frontend

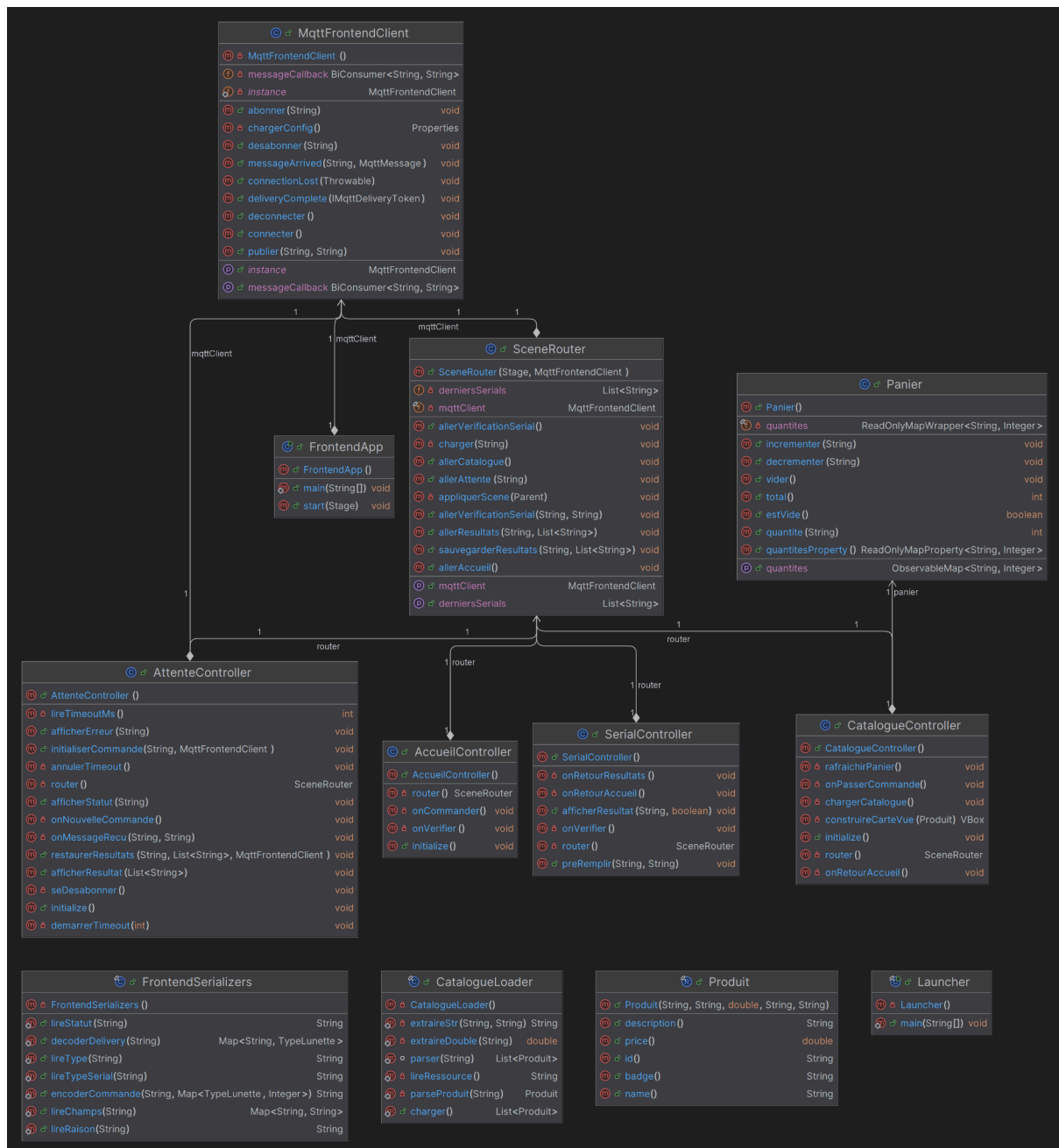


Figure 5 : Diagramme de classes de la partie frontend

2. Diagramme du serveur

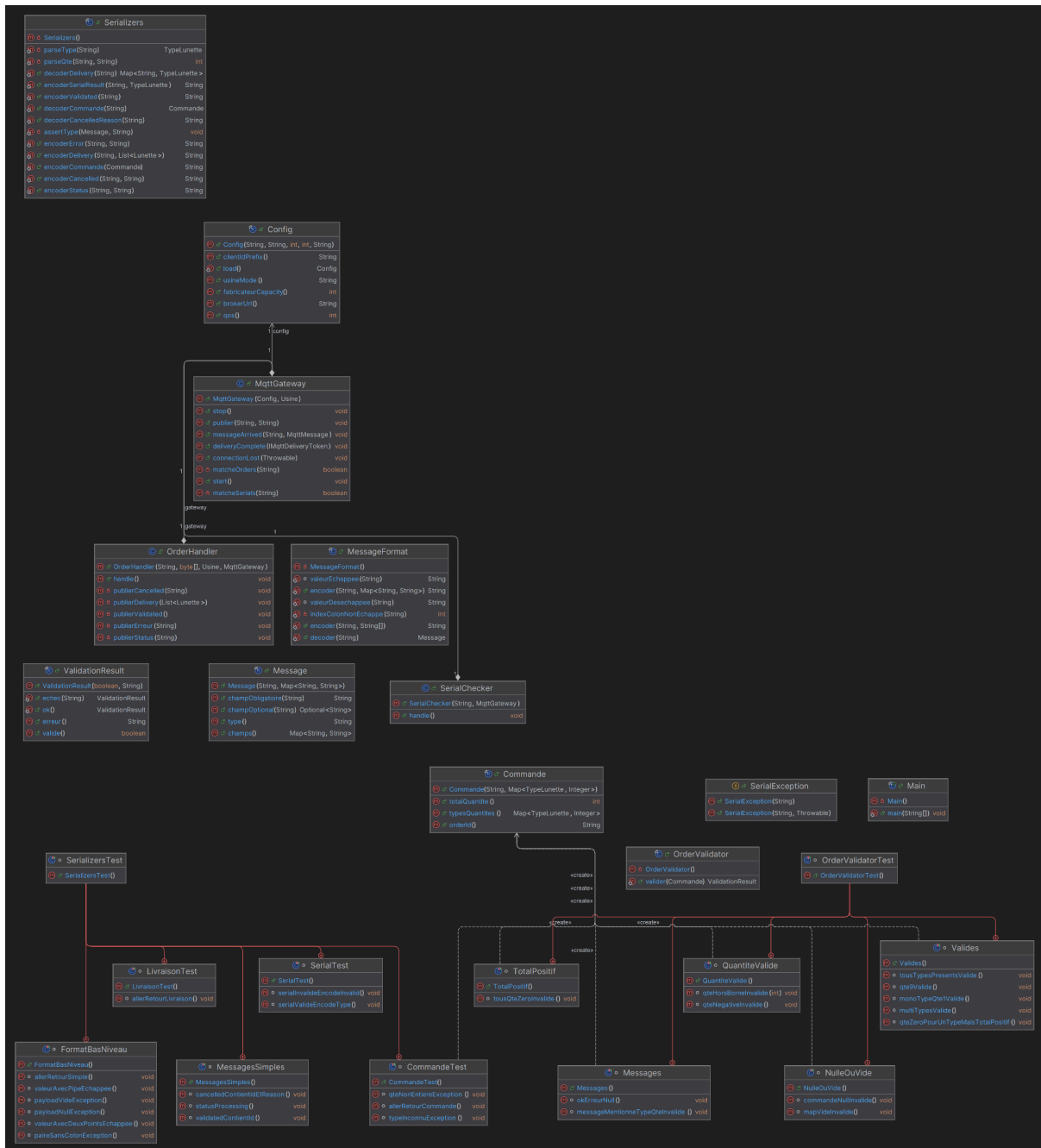


Figure 6 : Diagramme de classes de la partie serveur

3. Diagramme de l'usine

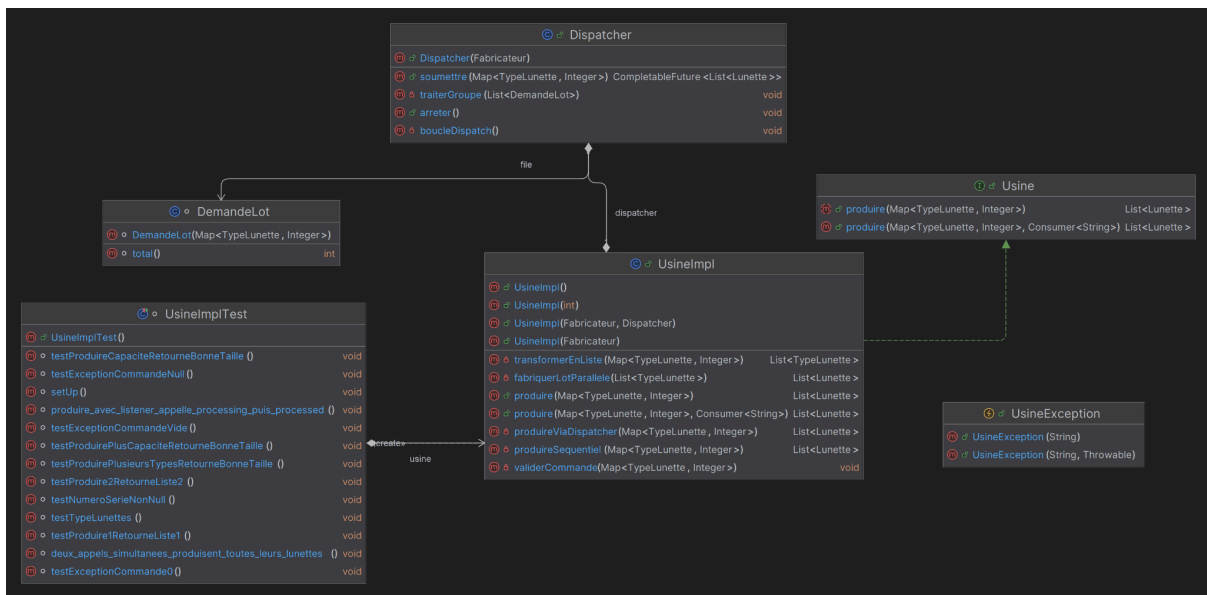


Figure 7 : Diagramme de classes de l'usine

4. Conception du backend

A. Explications générales

Notre backend est divisé en deux parties: le serveur MQTT et l'usine pour la production. Le serveur ne connaît pas les détails de fabrication, toute la fabrication est gérée par l'usine via son interface. L'usine ne connaît rien de MQTT et s'occupe uniquement de la production de lunettes.

B. L'usine

1. Interface Usine

L'interface usine contient deux méthodes. Initialement, il n'y avait qu'une seule méthode prenant en paramètre une `Map<TypeLunette, Integer>` qui permettait d'indiquer une commande à produire. Nous avons ajouté une version de cette méthode avec un paramètre supplémentaire de type `Consumer<String>` qui permet au serveur de publier au bon moment les statuts *processed* et *processing*.

2. Classe UsineImpl

La classe `UsineImpl`, supporte les deux modes de fonctionnements:

- Mode séquentiel : Chaque appel à `produire()` gère son propre lot. Les fabrications du même lot s'exécutent en parallèle grâce à un `ExecutorService` de taille fixée par celle de la machine. Les lots ne seront donc pas complets même si le total de commande pourrait faire un lot complet.
- Mode mutualisé : Un dispatcher est ajouté, la méthode `produire()` va faire appel au dispatcher pour regrouper les commandes dans le même lot selon la capacité de la machine.

3. Classe Dispatcher

Le dispatcher est utilisé dans le cas où plusieurs petites commandes pouvant être regroupées arrivent au même moment. Cela permet d'éviter d'attendre la fin d'une commande avant de traiter la suivante. Les commandes seront alors regroupées afin d'utiliser au mieux la capacité de la machine.

Exemple :

Si la machine a une capacité de 8 et que l'on a 3 commandes de 2 lunettes :

- En mode séquentiel : Les 3 commandes de 2 lunettes seront effectuées à la suite. Donc cela provoquerait un gaspillage de 6 places à chaque fois.
- En mode mutualisé : Le dispatcher va regrouper les commandes grâce à `configurer()` pour créer un seul lot de 6 lunettes qui sera fabriqué en une seule fois.

Le dispatcher utilise une `LinkedBlockingQueue` pour stocker les commandes en attente et un thread dédié pour filtrer les commandes compatibles entre elles.

C. Serveur

1. Classe Main

Une série d'étape se déclenche lorsque cette classe est lancée :

- Chargement de la configuration grâce à `config.load()`.
- Instanciation du Fabricateur (capacité fixée ou aléatoire si 0 indiqué).
- Instanciation du dispatcher si le mode de l'usine est définie sur mutualisée.
- Création de `UsineImpl`
- Démarrage de `MqttGateway`

2. Classe MqttGateway

Cette classe encapsule les méthodes qui servent à l'interaction avec le broker. Elle implémente l'interface `MqttCallback` de Paho afin de recevoir les messages. Elle possède une méthode `publier()` qui est utilisée par les autres classes pour émettre des réponses.

Lors de la réception d'un message, le routage est basé sur le topic:

- `orders/{uuid}` → création d'un `OrderHandler`
- `serials/{uuid}/check` → création d'un `SerialChecker`

Chaque commande est traitée dans un thread dédié, sans bloquer le thread de réception MQTT ni les autres commandes.

3. Classe OrderHandler

Il y a une instance de cette classe par commande, elle est exécutée dans un thread dédié. Le cycle est :

- Désérialisation du payload en Commande via `Serializers.decoderCommande()`
- Validation via `OrderValidator.valider()`
- Publication de `validated` ou `cancelled` selon le résultat
- Appel à `usine.produire(commande, listener)` où le listener publie `processing` puis `processed`
- Publication de `delivery` avec la liste des numéros de série
- Publication d'`error` en cas d'exception (`try/catch` global)

4. Classe SerialChecker

La vérification dans cette classe est synchrone et statique :

- Appel à Fabricateur.validateSerial(serial) (méthode statique, thread-safe)
- Publication du résultat sur serials/<serial> (type de lunette ou "invalid")

5. Classe Config

Cette classe comporte la méthode statique load() qui applique une stratégie de résolution pour le fichier properties :

- Chemin passé en -Dfabrique.config=/chemin/vers/config.properties
- Fichier config.properties dans le répertoire courant
- Fichier config.properties embarqué dans le jar

5. Conception du frontend

A. Explications générales

L'application JavaFx est divisée en quatre écrans, chacun possédant son fichier FXML ainsi qu'un contrôleur.

Ecran	Fichier FXML	Rôle
Accueil	accueil.fxml	Page d'accueil de l'application avec la navigation
Catalogue	catalogue.fxml	Liste des lunettes, panier, passage de commandes
Attente	attente.fxml	Suivi de la fabrication (status + barre de progression) ainsi que l'affichage des numéros de série une fois la commande reçue. Les numéros de séries sont des liens cliquables qui redirigent vers l'écran Vérification
Vérification	serial.fxml	Vérification des numéros de série

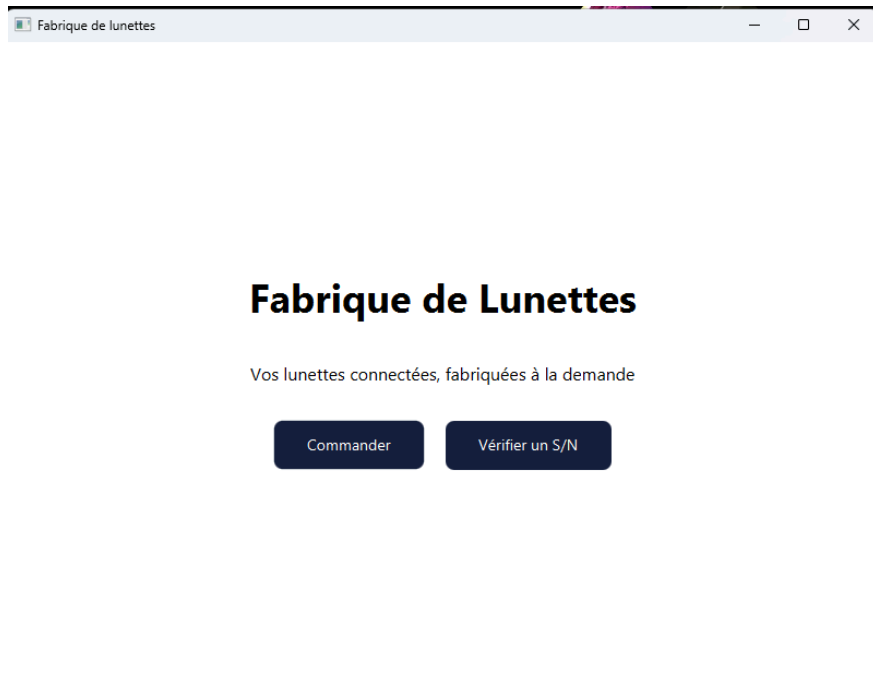


Figure 8 : JavaFx de la page d'accueil

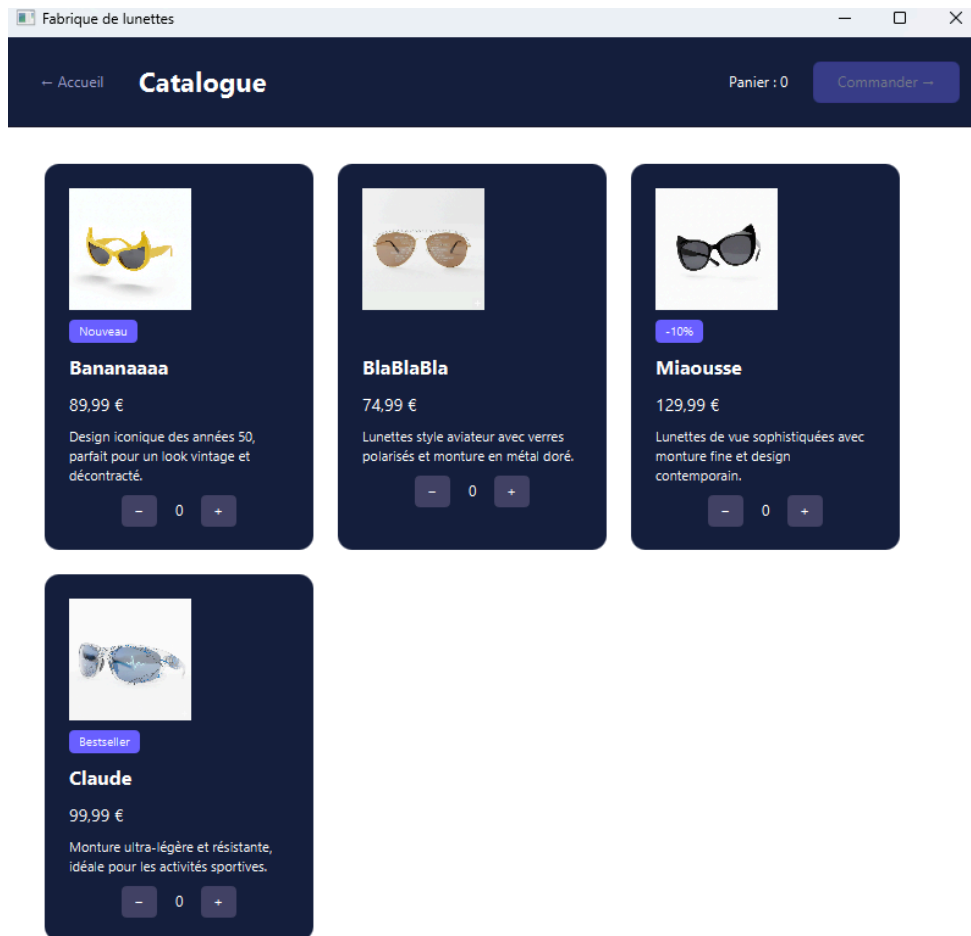


Figure 9 : JavaFx de la page de catalogue

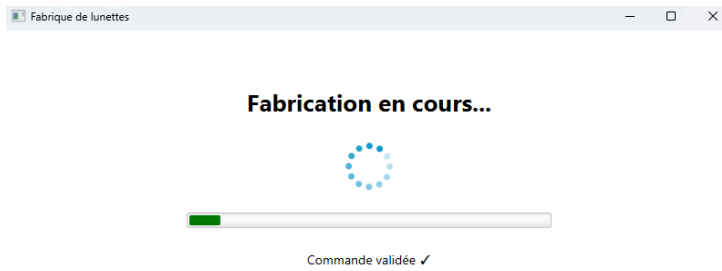


Figure 10 : JavaFx de la page d'attente en cours de chargement

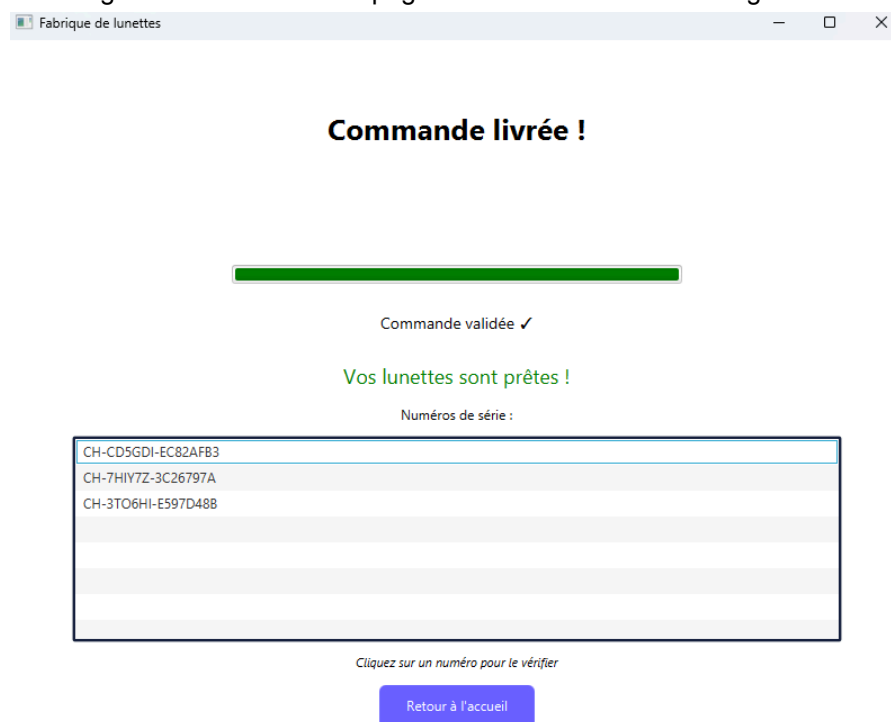


Figure 11 : JavaFx de la page d'attente quand la livraison est faite

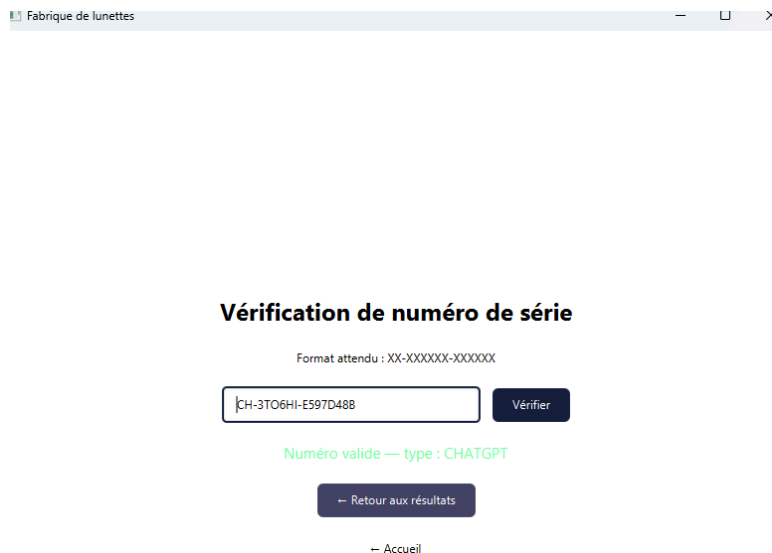


Figure 12 : JavaFx de la page de vérification du numéro de série

B. Navigation entre les écrans

La classe `SceneRouteur` sert à la navigation. Elle est instanciée au démarrage et passée à chaque contrôleur via `setUserData()`. Chacun des contrôleurs accède au routeur via la méthode `getScene().getRoot().getUserData()` et appelle ensuite la route qu'il souhaite emprunter avec `route.aller<NomRoute>()`. Cela permet d'avoir un couplage direct entre les contrôleurs et facilite l'ajout d'autres services en passant simplement par le routeur.

C. MQTT côté frontend

La classe `MqttFrontendClient` est un singleton pour gérer la connexion Paho. Elle a une API simple:

- `connecter()` → une seule fois au démarrage de l'application
- `publier(topic, payload)` → émettre un message
- `abonner(topic)` → s'abonner à un topic
- `desabonner(topic)` → se désabonner à un topic
- `setMessageCallback(callback)` → installer un callback appelé à chaque message reçu

Le patron de callback unique est adapté à notre situation car nous n'avons qu'un seul écran à la fois qui a besoin de recevoir les messages entrants. Chaque contrôleur active son callback lorsque l'on initialise l'écran et le retire lorsque l'on change d'écran.

D. Mise à jour de l'interface depuis le thread MQTT

Pour mettre à jour notre interface dans un contexte multi-threads, il est primordial d'utiliser la méthode `Platform.runLater()` pour forcer l'utilisation du thread JavaFX pour générer l'interface. En effet, sans cette méthode nous serions confrontés à une erreur du type *Exception in thread "pool-1-thread-1" java.lang.IllegalStateException: Not on FX application thread.*

E. Gestion du panier

La classe `Panier` possède un `ObservableMap<String, Integer>`. Le contrôleur du catalogue installe un listener qui permet d'activer le bouton de commande lorsque le panier n'est plus vide et de désactiver le bouton sinon.

F. Gestion du timeout sur l'écran d'attente

Si pour différentes raisons l'usine ne répond pas, l'utilisateur ne doit pas rester bloqué indéfiniment sur la page d'attente. Un `ScheduleExecutorService` déclenche un message d'erreur.

Le chrono est annulé dès la réception du premier message lié à la commande et il redémarre à chaque étape. Cela permet de prendre en compte les cas où le backend est démarré mais s'arrête en cours de production.

G. Catalogue dynamique

Le catalogue des lunettes est défini dans le fichier `products.json` présent dans le jar. La classe `CatalogueLoader` parse le JSON. Le format attendu est figé :

```
[
  {
    "id": "claudio", "name": "Claude", "price": 99.99,
    "badge": "Bestseller",
    "description": "Monture ..."
  },
  ...
]
```

L'id du produit est le type de lunettes écrit en minuscule.

6. Protocole MQTT

A. Conventions de nommage des topics

Les topics sont organisés en deux catégories principales :

- `orders/{uuid}` — topic d'une commande.
- `serials/{serial}` — topic d'une vérification de numéro de série.

B. Identifiants uniques

Le `{uuid}` dans les topics est un UUID v4 généré côté frontend via `java.util.UUID.randomUUID()`. Cette méthode garantit l'unicité des id de commandes.

C. Tableau récapitulatif des topics

Topic	Sens	Description
orders/{uuid}	Client → Usine	Passage d'une commande (payload order)
orders/{uuid}/validated	Usine → Client	Commande valide, sera fabriquée
orders/{uuid}/cancelled	Usine → Client	Commande invalide, transaction annulée
orders/{uuid}/status	Usine → Client	Changement d'état (processing / processed)
orders/{uuid}/delivery	Usine → Client	Livraison avec les numéros de série
orders/{uuid}/error	Usine → Client	Erreur de fabrication, transaction terminée
serials/{serials}/check	Client → Usine	Vérification d'un numéro de série
serials/{serial}	Usine → Client	Réponse : type de lunettes ou "invalid"

D. QoS et rétention

Nous utilisons QoS 1 afin de garantir qu'un message est délivré au moins une fois au destinataire. Cela permet de ne pas perdre de commande ni de livraison. De plus, nous n'avons pas de rétention afin que les clients qui s'abonnent ne voient pas les commandes passées.

E. Cas d'erreur et résilience

Nous avons identifié 3 cas pouvant générer des erreurs.

1. Perte de connexion broker

Nous avons configuré la reconnection automatique de Paho. Ainsi, en cas de connexion perdue, le client MQTT essaie de se reconnecter. De plus, les abonnements sont restaurés automatiquement après la reconnexion et les messages qui n'ont pas été acquittés sont retransmis.

2. Backend indisponible

Nous avons mis en place un système de timeout afin que le client abandonne sa commande dans le cas où le backend ne serait pas disponible. Cela évite de rester bloqué indéfiniment sur l'écran d'attente.

3. Payload malformé

Si un payload ne respecte pas notre sérialisation, la désérialisation va lever une `SerialException`. Celle-ci sera gérée dans `OrderHandler` qui affichera un message d'erreur sur l'interface utilisateur.

F. Diagrammes de séquences

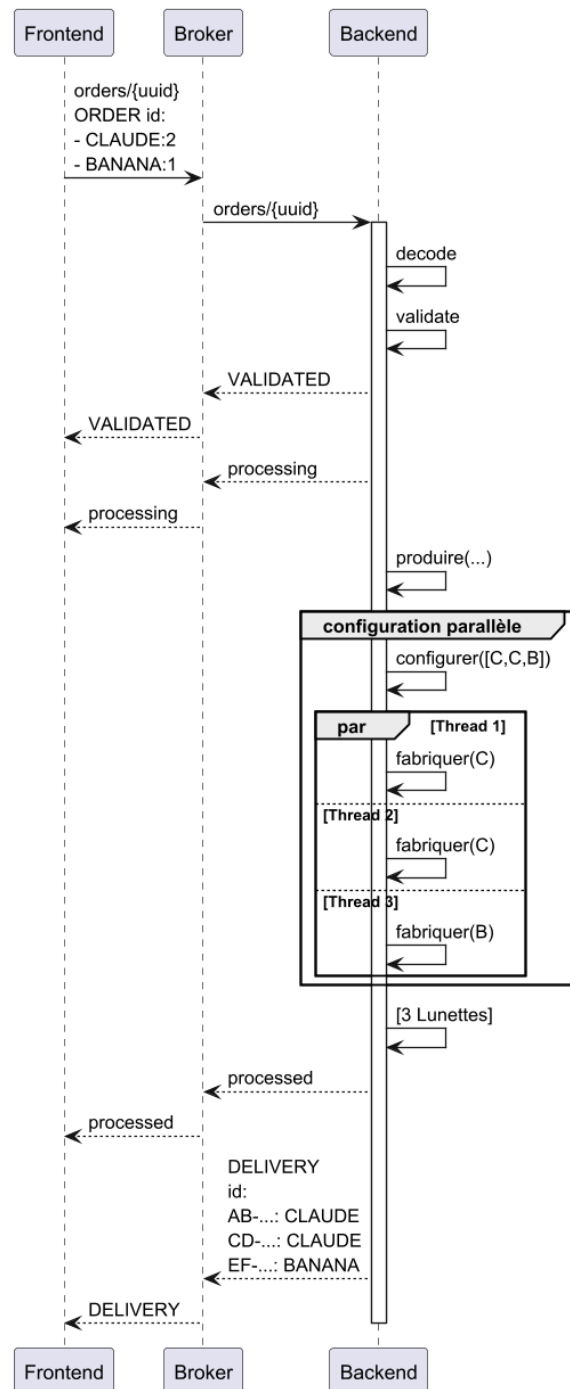


Figure 13 : Commande validée et livrée

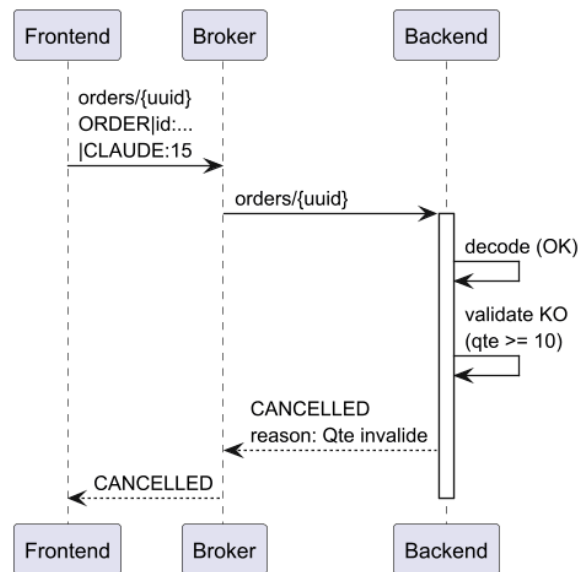


Figure 14 : Commande invalide

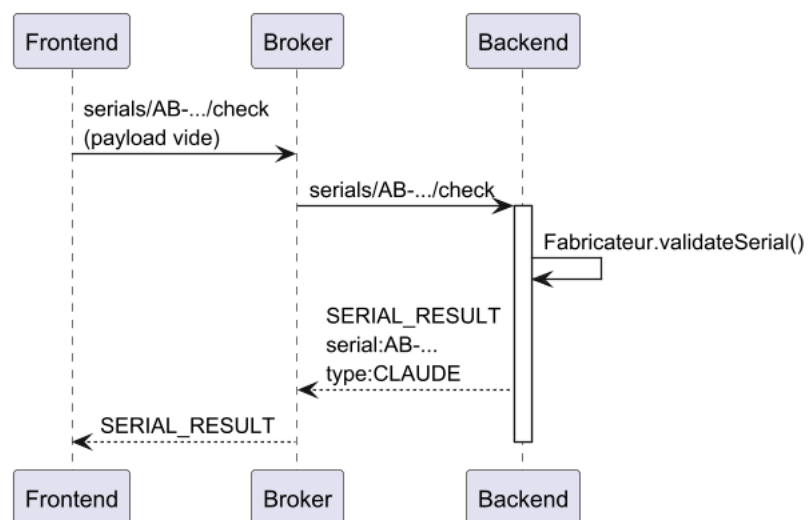


Figure 15 : Vérification d'un numéro de série

7. Format de Sérialisation

A. Grammaire

Nous avons créé cette grammaire simple pour notre format de sérialisation :

message ::= TYPE | paire | ...

paire ::= cle : valeur

Le caractère '[' nous permet de séparer le type et les paires tandis que le caractère ':' nous permet quant à lui de séparer la clé et la valeur dans chacune des paires. Il faut impérativement échapper ces caractères.

B. Types de messages

Type	Sens	Description
ORDER	Client → Usine	Commande (clés = nom de TypeLunette, valeurs = quantité)
VALIDATED	Usine → Client	Commande validée (id uniquement)
CANCELLED	Usine → Client	Commande annulée (id + reason)
STATUS	Usine → Client	Changement d'état (id + status)
DELIVERY	Usine → Client	Livraison (id + serial:type pour chaque lunette)
ERROR	Usine → Client	Erreur en cours de traitement (id + reason)
SERIAL_RESULT	Usine → Client	Résultat de vérification S/N (serial + type ou "invalid")

C. Exemples annotés

Voici trois exemples d'application de notre type de sérialisation.

1. Passage de commande

`ORDER|id:550e8400-e29b-41d4-a716-446655440000|CLAUDE:2|BANANA:1`

- Type : ORDER
- Champ id : identifiant unique de la commande (UUID)
- Champs CLAUDE et BANANA : quantités demandées pour chaque type de lunettes

2. Livraison

`DELIVERY|id:550e8400-...|AB-123456-789012:CLAUDE|CD-654321-210987:CLAUDE|EF-...:BANANA`

- Inversion conceptuelle : les clés sont ici les numéros de série, les valeurs sont les types
- Cette inversion est rendue possible par le fait qu'un numéro de série est unique.

3. Erreur avec caractère réservé dans le message

`ERROR|id:550e...|reason:erreur \: machine indisponible`

- Le \: dans reason est l'échappement du caractère :
- À la lecture, le décodeur restaure "erreur : machine indisponible"

D. Implémentation

Notre format de sérialisation est implémenté dans le package `fr.fabrique.serveur.serial`. Il est constitué de trois classes :

- `MessageFormat` → sert à encoder/décoder
- `Message` → record représentant un message décodé (type + champs)
- `Serializers` → regroupés par métier. Une méthode par type (`encoderCommande`, `encoderDelivery`, ...)

Certaines des méthodes de ce type de sérialisation sont dupliquées dans le frontend. Cette duplication permet l'indépendance du frontend.

8. Tests

En ce qui concerne les tests, nous nous sommes concentré sur les fonctionnalités les plus importantes c'est -à -dire, validation des commandes, sérialisation, parallélisation. Nous n'avons pas réalisé de tests sur l'interface. Les tests utilisent JUnit 5 sans bibliothèques externes.

A. Les tests réalisés

Fichier	Couverture
OrderValidatorTest	11 tests sur les règles métiers
SerializersText	13 test sur la méthode de sérialisation
UsineImplTest	11 tests sur la mise en lot et la parallélisation
DispatcherTest	4 tests sur la mutualisation

Certains tests peuvent être assez lent, cela vient du fait qu'ils utilisent le vrai Fabricateur avec la méthode fabriquer(). Quelques secondes de délai sont faites de manière volontaire pour simuler un temps de production réel.

B. Test non implémentés

De nombreux autres test auraient pus être implémenté, pour rendre le projet plus robuste:

- Tests d'intégration avec un broker Mosquitto.
- Tests du frontend via TestFx sur l'interface, la navigation...
- Tests plus poussés sur le dispatcher, par exemple en testant de nombreuses commandes simultanément.
- Tests MQTT, déconnexion et reconnexion lors du traitement.

9. CI/CD et publication

Nous n'avons pas réussi à faire fonctionner les github action. Nous allons tout de même vous présenter ce que nous voulions faire.

A. Workflows

Nous avons essayé de faire 4 workflow : 1 workflow CI, 2 workflows pour générer automatiquement les fichiers jar lors d'une release (un pour le frontend et un pour le backend) et 1 workflow pour publier le package usine. Comme dit précédemment, nous n'avons pas réussi à les exécuter.

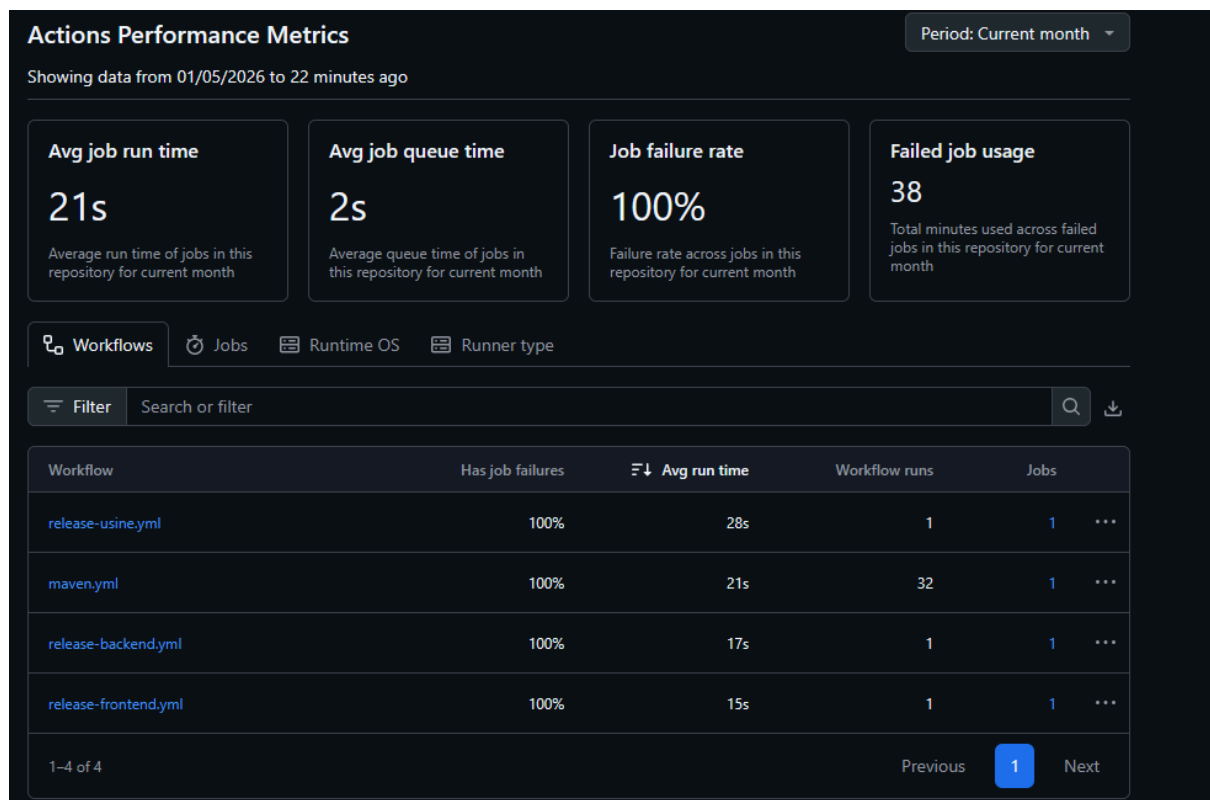


Figure 16 : Rapport github actions

B. Secrets

Nous avons créé des secrets pour que les github action puissent accéder au dépôt maven privé contenant le fabricant.

FABRICATEUR_USERNAME

FABRICATEUR_TOKEN

10. Questions du sujet

A. Comment gérer l'absence d'usine connectée au bus pour produire les lunettes ?

Le frontend publie une commande sur orders/{uuid}. Si il n'y a pas de backend connecté au broker, le message est envoyé sans destinataire et est donc perdu. L'utilisateur sera donc bloqué sur l'écran d'attente et n'aura aucun retour.

La solution implémentée dans notre projet est d'avoir un timeout côté frontend. Le contrôleur AttenteController démarre ScheduledExecutorService avec un délai configurable. Si aucun message lié à la commande n'est reçu avant la fin du chrono, un message d'erreur apparaît. Le chrono est annulé à la réception du premier message lié à la commande.

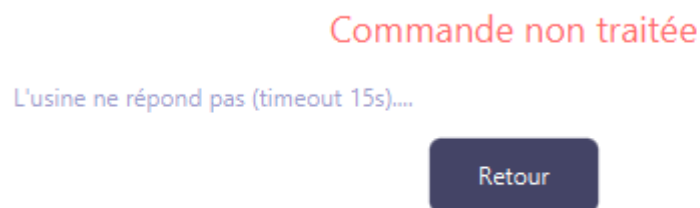


Figure 17 : Message d'erreur à la fin du timer

B. Comment gérer plusieurs commandes en parallèle pour un même client?

Actuellement, chaque instance de l'application est limitée à une seule commande en cours. L'utilisateur passe une commande, attend la livraison et ne peut pas faire de nouvelle commande sans l'avoir reçue.

Pour gérer ce problème, il faudrait:

- Changer la classe `MqttFrontendClient` qui fait un callback unique. Il faudrait faire un dispatcher par topic.
- Stocker la liste des commandes de manière observable et partagée entre tous les écrans.
- Changer le visuel pour pouvoir suivre plusieurs commandes sur l'écran d'attente et continuer de faire des commandes.
- Adapter les abonnements MQTT.

Il ne devrait pas y avoir de modifications à effectuer côté backend étant donné que le serveur traite déjà chaque commande dans des thread indépendants.

C. Que faut-il modifier pour pouvoir gérer plusieurs usines ?

L'architecture actuelle permet d'avoir une seule instance du backend. Si on en lance plusieurs, le problème serait qu'étant donné qu'elles écoutent toutes le topic `orders/+`, elles recevront toutes des commandes. Les commandes seront donc fabriquées autant de fois que d'instance du backend.

Afin d'éviter que toutes les usines reçoivent les commandes, il faudrait qu'on gère une notion d'id pour les usines et que nous ayons un service jouant le rôle de load balancer entre les différentes usines. Ainsi lorsqu'une usine est créée elle vient s'inscrire dans le load balancer. De plus, les commandes ne seront plus passées aux usines mais au load balancer qui se chargera d'envoyer les commandes dans les usines qui se sont inscrites auprès de lui.

11. Répartition du travail

Personne	Domaine	Tâches
Patrick	Backend	Serveur
Victor	Backend	Usine
Raphaël	Build	Squelette du projet, GitHub Action, Sérialisation, README ...
Mélissandre	Frontend	JavaFx, navigation, client MQTT côté UI

12. Conclusion

Pour conclure, nous avons réussi à répondre à quasiment toutes les exigences du sujet. Nous n'avons simplement pas réussi à mettre en place de CI/CD avec les githubs actions.

Concernant les difficultés rencontrées, le point où nous avons eu le plus de difficulté est bien avant les github actions. Nous avons essayé de suivre le tutoriel disponible sur github mais même avec un projet nous n'avons jamais réussi à avoir un job validé.

Ce projet pourrait évoluer et être enrichi par de nouvelles fonctionnalités comme la possibilité d'avoir accès à un historique des commandes passées. On pourrait également faire davantage de tests par exemple sur la partie frontend.